Next.js 15

TypeScript

Graph API v1.0

Firebase Auth

Tailwind CSS

# Integracion Microsoft OneDrive en ArchiFlow

Documento tecnico de implementacion para la integracion de Microsoft Graph API con la plataforma ArchiFlow. Incluye gestion de archivos, galeria de fotos de proyectos, explorador de documentos y almacenamiento en la nube directamente desde la aplicacion.

PROYECTO

ArchiFlow - Plataforma de Gestion para Arquitectura

TECNOLOGIAS

Next.js / TypeScript / Microsoft Graph API / Firebase Auth

FECHA

Abril 2026

# Tabla de Contenidos


1. Resumen Ejecutivo 3

2. Arquitectura del Sistema 3

2.1 Flujo de Autenticacion OAuth 2.0 4

3. Modelo de Datos 5

3.1 Coleccion onedrive_files 5

3.2 Coleccion onedrive_folders 6

4. Rutas API de Next.js 7

4.1 Gestion de Archivos 7

4.2 Galeria de Fotos 7

4.3 Ejemplo de Implementacion: Listar Archivos 8

5. Componentes de Interfaz de Usuario 9

5.1 Explorador de Archivos 9

5.2 Galeria de Fotos del Proyecto 9

5.3 Componente de Subida de Archivos 9

6. Permisos y Seguridad 10

7. Scopes Requeridos de Microsoft Graph API 11

8. Estrategia de Implementacion por Fases 11

9. Consideraciones Tecnicas 12

9.1 Gestion de Tokens y Refresh 12

9.2 Limitaciones de la API 13

9.3 Rendimiento y Cache 13

10. Resumen de Dependencias y Prerequisitos 13

# 1. Resumen Ejecutivo

El presente documento detalla el plan de implementacion tecnica para integrar Microsoft OneDrive como sistema de almacenamiento principal de archivos y galeria de fotos en la plataforma ArchiFlow. ArchiFlow es una aplicacion de gestion de proyectos orientada al sector de la arquitectura y el diseno de interiores, construida con Next.js 15, TypeScript, Tailwind CSS y Firebase como backend. La integracion con OneDrive permitira a los usuarios almacenar, gestionar y compartir documentos y fotografias de proyectos directamente desde la aplicacion, aprovechando la infraestructura en la nube de Microsoft y la autenticacion OAuth 2.0 ya existente a traves de Firebase Auth.

La decision de integrar OneDrive responde a una necesidad operativa concreta: los profesionales de la arquitectura manejan grandes volumenes de archivos (planos, renders, fotografias de obra, presupuestos, contratos) que actualmente requieren gestion manual en sistemas externos. Al integrar esta funcionalidad directamente en ArchiFlow, se elimina la friccion de cambiar entre aplicaciones, se reduce el riesgo de desincronizacion de archivos y se ofrece una experiencia unificada que aumenta la productividad del equipo de trabajo. Ademas, la integracion aprovecha los 7 roles de usuario existentes en ArchiFlow para controlar el acceso a los archivos de cada proyecto de manera granular y segura.

El alcance de esta implementacion cubre cuatro modulos principales: el explorador de archivos con navegacion por carpetas, la galeria de fotos del proyecto con thumbnails y vista previa, el sistema de subida y descarga con soporte para archivos grandes, y la sincronizacion de permisos basada en los roles existentes de la aplicacion. El documento incluye la arquitectura del sistema, los modelos de datos propuestos, las rutas API necesarias, los componentes de interfaz de usuario y la estrategia de implementacion por fases con prioridades y estimaciones de esfuerzo.

# 2. Arquitectura del Sistema

La arquitectura de la integracion sigue el patron de tres capas que ya utiliza ArchiFlow: un frontend con componentes React (Next.js App Router), una capa intermedia de API Routes del servidor Next.js, y los servicios externos (Firebase Auth y Microsoft Graph API). La comunicacion entre el frontend y las API Routes se realiza a traves de fetch con el token de acceso de Firebase, mientras que las API Routes actuan como intermediarias seguras que reenvian las solicitudes a Microsoft Graph API utilizando el token OAuth de Microsoft obtenido durante la autenticacion del usuario.

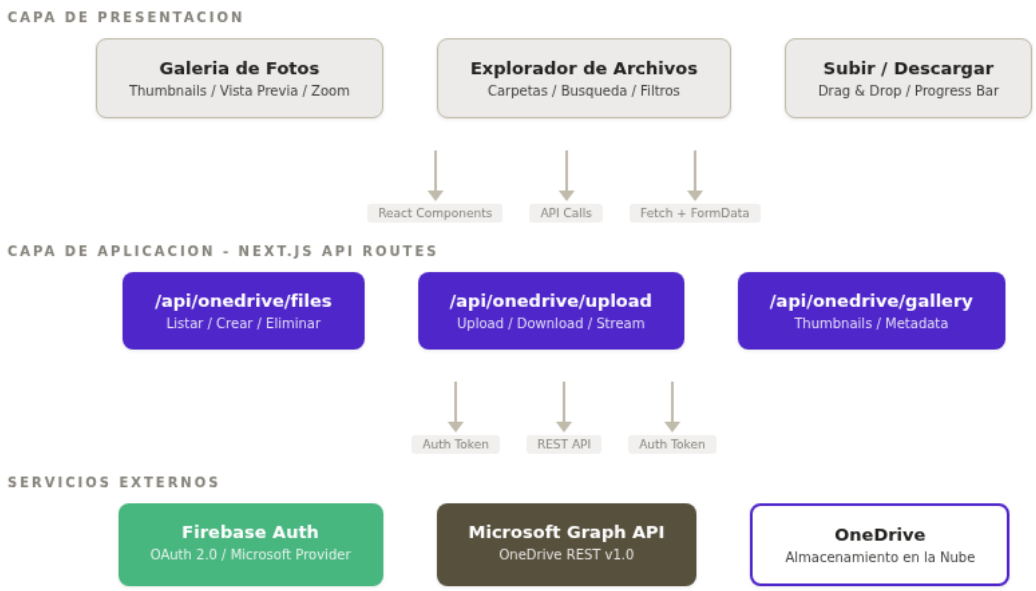


Figura 1: Arquitectura de tres capas para la integracion OneDrive en ArchiFlow.

La capa de presentacion esta compuesta por componentes React que se comunican con las API Routes de Next.js. Estas rutas actuan como proxy seguro, ya que el token de acceso de Microsoft nunca se expone al navegador del cliente. La capa de aplicacion gestiona la logica de negocio, incluyendo la validacion de permisos, la transformacion de respuestas de la API y el manejo de errores. La capa de servicios externos incluye Firebase Auth (para la autenticacion y generacion de tokens OAuth) y Microsoft Graph API (para las operaciones de lectura, escritura y gestion de archivos en OneDrive).

## 2.1 Flujo de Autenticacion OAuth 2.0

ArchiFlow ya utiliza Firebase Auth con el proveedor de Microsoft (MicrosoftAuthProvider). Cuando un usuario inicia sesion con su cuenta de Microsoft, Firebase genera un token de acceso OAuth que incluye los permisos delegados solicitados en el scope. Este token es el que se utiliza para autenticar las llamadas a Microsoft Graph API desde las API Routes del servidor. El flujo completo es el siguiente: el usuario hace clic en "Iniciar sesion con Microsoft", Firebase redirige al usuario a la pagina de consentimiento de Microsoft, el usuario acepta los permisos, Firebase recibe el codigo de autorizacion y lo intercambia por tokens (access_token y refresh_token), y finalmente el token de acceso se almacena de forma segura en Firebase Auth para ser utilizado en llamadas posteriores a Graph API.

| Paso | Accion | Componente | Resultado |
|---|---|---|---|
| 1 | Usuario inicia sesion con Microsoft | Firebase Auth Frontend | Redireccion a MS Login |
| 2 | Consentimiento de permisos | Microsoft OAuth | Authorization Code |
| 3 | Intercambio de codigo por tokens | Firebase Backend | Access Token + Refresh |
| 4 | Almacenamiento seguro de tokens | Firebase Auth SDK | Token en sesion de usuario |
| 5 | Llamada a Graph API con token | Next.js API Route | Datos de OneDrive |

Tabla 1: Flujo de autenticacion OAuth 2.0 para Microsoft Graph API.

# 3. Modelo de Datos

La integracion con OneDrive no requiere cambios significativos en el esquema de base de datos existente de ArchiFlow (Prisma + Firestore). Los archivos se almacenan en OneDrive y los metadatos relevantes se sincronizan con Firestore para permitir busquedas rapidas y mantener la relacion entre archivos y proyectos. A continuacion se definen las colecciones de Firestore que gestionan la relacion entre los archivos de OneDrive y los proyectos de ArchiFlow.

## 3.1 Coleccion onedrive_files

Esta coleccion almacena los metadatos de los archivos almacenados en OneDrive que estan asociados a un proyecto especifico de ArchiFlow. Cada documento representa un archivo individual y contiene una referencia al itemId de OneDrive para operaciones posteriores (descarga, eliminacion, actualizacion de metadatos). La coleccion permite consultar archivos por proyecto, por tipo, o por categoria sin necesidad de hacer llamadas adicionales a la API de Microsoft, lo que mejora significativamente el rendimiento de la interfaz de usuario.

| Campo | Tipo | Descripcion | Ejemplo |
|---|---|---|---|
| id | string | ID unico del registro en Firestore | auto-generated |
| projectId | string (ref) | Referencia al proyecto en ArchiFlow | projects/abc123 |
| driveItemId | string | ID del archivo en OneDrive (Graph API) | 01ABCD... |
| name | string | Nombre del archivo original | plano_planta.pdf |
| mimeType | string | Tipo MIME del archivo | application/pdf |
| size | number | Tamano en bytes | 2048576 |
| category | string | Categoria del archivo | planos / fotos / contratos |
| thumbnailUrl | string? | URL del thumbnail (fotos) | https://... |
| uploadedBy | string (ref) | Referencia al usuario que subio | users/xyz789 |
| createdAt | timestamp | Fecha de creacion del registro | 2026-04-11... |
| parentId | string? | ID del folder padre en OneDrive | 01PARENT... |

Tabla 2: Estructura de la coleccion onedrive_files en Firestore.

## 3.2 Coleccion onedrive_folders

Esta coleccion almacena la estructura de carpetas que se crea automaticamente en OneDrive para cada proyecto de ArchiFlow. Cuando se crea un nuevo proyecto, se genera una carpeta raiz con subcarpetas predefinidas (planos, fotos, contratos, presupuestos, otros). Esta estructura estandarizada permite mantener el orden de los archivos sin depender de que los usuarios la creen manualmente, reduciendo errores y garantizando consistencia en la organizacion de documentos entre proyectos.

| Subcarpeta | Uso | Tipos permitidos |
|---|---|---|
| planos/ | Planos arquitectonicos y de ingenieria | .pdf, .dwg, .dxf, .svg |
| fotos/ | Fotografias de obra, renders y moodboards | .jpg, .png, .heic, .webp |
| contratos/ | Contratos y documentos legales | .pdf, .docx, .xlsx |
| presupuestos/ | Presupuestos y cotizaciones | .pdf, .xlsx, .csv |
| otros/ | Documentos generales del proyecto | Cualquier tipo |

Tabla 3: Estructura estandarizada de carpetas por proyecto en OneDrive.

# 4. Rutas API de Next.js

Las rutas API de Next.js actuan como intermediarias seguras entre el frontend de React y Microsoft Graph API. Todas las rutas validan el token de autenticacion del usuario, verifican que tenga permisos sobre el proyecto solicitado, y reenvian la solicitud a Graph API con el token OAuth de Microsoft. Este patron de proxy garantiza que las credenciales de acceso nunca se exponen en el navegador del cliente y permite implementar logica de autorizacion basada en los roles existentes de ArchiFlow.

## 4.1 Gestion de Archivos

| Ruta | Metodo | Endpoint Graph API | Descripcion |
|---|---|---|---|
| /api/onedrive/files | GET | GET /me/drive/items/{id}/children | Listar archivos de una carpeta |
| /api/onedrive/files | POST | PUT /me/drive/items/{id}:/{filename}/content | Subir archivo a carpeta |
| /api/onedrive/files/[id] | DELETE | DELETE /me/drive/items/{id} | Eliminar archivo de OneDrive |
| /api/onedrive/files/[id] | PATCH | PATCH /me/drive/items/{id} | Renombrar o mover archivo |
| /api/onedrive/files/[id]/download | GET | GET /me/drive/items/{id}/content | Descargar archivo binario |
| /api/onedrive/search | GET | GET /me/drive/root/search(q=...) | Buscar archivos por nombre |

Tabla 4: Rutas API para gestion de archivos en OneDrive.

## 4.2 Galeria de Fotos

| Ruta | Metodo | Descripcion |
|---|---|---|
| /api/onedrive/gallery/[projectId] | GET | Obtener lista de fotos del proyecto con thumbnails |
| /api/onedrive/gallery/[id]/thumbnail | GET | Generar y obtener thumbnail de tamaño personalizado |
| /api/onedrive/gallery/batch | POST | Carga multiple de fotos (batch upload) |

Tabla 5: Rutas API para la galeria de fotos.

## 4.3 Ejemplo de Implementacion: Listar Archivos

A continuacion se presenta un ejemplo detallado de la implementacion de la ruta API para listar archivos de una carpeta de OneDrive. Este ejemplo ilustra el patron general que se aplicara a todas las rutas de la integracion: validacion de autenticacion, verificacion de permisos, llamada a Graph API, transformacion de la respuesta y manejo de errores.

```
// src/app/api/onedrive/files/route.ts
import { NextRequest, NextResponse } from 'next/server';

export async function GET(req: NextRequest) {
const { searchParams } = new URL(req.url);
const folderId = searchParams.get('folderId') || 'root';
const projectId = searchParams.get('projectId');
const accessToken = req.headers.get('Authorization')
?.replace('Bearer ', '');

if (!accessToken || !projectId) {
return NextResponse.json(
{ error: 'Autenticacion requerida' }, { status: 401 }
);
}

// Verificar permisos del usuario sobre el proyecto
// (validar rol en Firestore usando projectId)

const res = await fetch(
`https://graph.microsoft.com/v1.0/me/drive/items/${folderId}/children`
?top=50&$orderby=name&$select=id,name,size,mimeType,
lastModifiedDateTime,thumbnails`,
{
headers: { Authorization: `Bearer ${accessToken}` },
}
);

if (!res.ok) {
return NextResponse.json(
{ error: 'Error en Graph API' }, { status: res.status }
);
}

const data = await res.json();
return NextResponse.json(data.value);
}
```

# 5. Componentes de Interfaz de Usuario

La interfaz de usuario para la integracion de OneDrive se construye utilizando los componentes de shadcn/ui ya disponibles en ArchiFlow, combinados con Tailwind CSS para el estilizado. Los componentes se organizan en cuatro modulos principales que cubren las funcionalidades de exploracion de archivos, galeria de fotos, carga de archivos y vista previa de documentos. Cada componente esta disenado para funcionar de forma responsiva y mantener la consistencia visual con el resto de la aplicacion, aprovechando el sistema de temas (claro/oscuro) y la internacionalizacion existente.

## 5.1 Explorador de Archivos

El explorador de archivos es el componente central de la integracion. Ofrece una experiencia similar a la de un gestor de archivos tradicional, con navegacion por carpetas, vista de lista y vista de cuadricula (grid), ordenamiento por diferentes criterios (nombre, fecha, tamano) y busqueda por nombre. Se implementa como un componente servidor de Next.js que recibe los datos de la API Route y los renderiza de forma eficiente con carga progresiva. La navegacion por carpetas utiliza un componente de breadcrumbs que muestra la ruta actual y permite volver a carpetas padre con un solo clic. El explorador soporta acciones contextuales (clic derecho o menu de tres puntos) para renombrar, mover, eliminar y descargar archivos individuales o seleccion multiple.

## 5.2 Galeria de Fotos del Proyecto

La galeria de fotos muestra las imagenes almacenadas en la subcarpeta "fotos/" del proyecto en OneDrive utilizando thumbnails generados por la API de Microsoft. El componente utiliza un layout de cuadricula responsive (masonry grid) que se adapta al tamano de pantalla del dispositivo. Al hacer clic en una foto, se abre un modal de vista previa con navegacion entre imagenes (anterior/siguiente), zoom, y la opcion de descargar la imagen en resolucion original. Los thumbnails se cargan de forma diferida (lazy loading) utilizando Intersection Observer para optimizar el rendimiento cuando hay cientos de fotos en un proyecto. La galeria tambien soporta la seleccion multiple para operaciones batch (descargar seleccionadas, mover a otra carpeta, eliminar).

## 5.3 Componente de Subida de Archivos

El componente de subida de archivos soporta drag-and-drop y seleccion de archivos desde el explorador del sistema operativo. Incluye una barra de progreso que muestra el porcentaje de carga completado para cada archivo, soporte para carga simultanea de multiples archivos con limite configurable, y validacion de tipos de archivo basada en la carpeta destino (por ejemplo, solo imagenes en la carpeta de fotos, solo PDFs en la carpeta de contratos). Para archivos mayores a 4MB, el componente utiliza la API de carga por partes (upload session) de Microsoft Graph, que permite reanudar cargas interrumpidas y mostrar un

progreso mas preciso. El componente tambien muestra previews instantaneas de las imagenes seleccionadas antes de la subida, permitiendo al usuario verificar que los archivos correctos han sido seleccionados.

# 6. Permisos y Seguridad

La seguridad de la integracion se basa en tres capas de proteccion: la autenticacion de Firebase Auth que verifica la identidad del usuario, los roles de ArchiFlow que determinan las acciones permitidas sobre los archivos de un proyecto, y los permisos delegados de Microsoft Graph API que definen el alcance de las operaciones sobre OneDrive. Esta estrategia de defensa en profundidad garantiza que un usuario no autorizado no pueda acceder a los archivos de un proyecto, incluso si conoce el ID del archivo en OneDrive, ya que la validacion se realiza tanto en el servidor de Firebase como en las API Routes de Next.js antes de reenviar cualquier solicitud a Microsoft Graph.

| Rol ArchiFlow | Ver Archivos | Subir | Eliminar | Descargar |
|---|---|---|---|---|
| Administrador | Si | Si | Si | Si |
| Arquitecto | Si | Si | Si | Si |
| Director de Obra | Si | Si | No | Si |
| Diseñador | Si | Si | No | Si |
| Cliente | Si | No | No | Si |
| Proveedor | Limitado | Si* | No | Propio |
| Invitado | No | No | No | No |

Tabla 6: Matriz de permisos por rol de ArchiFlow para operaciones de archivos.

Los permisos de Microsoft Graph API necesarios para la integracion se configuran en el portal de Azure Active Directory al registrar la aplicacion. Los scopes minimos requeridos son: Files.ReadWrite.All (para operaciones de lectura y escritura sobre archivos), Files.Read.All (para lectura de metadatos), y User.Read (para obtener informacion del perfil del usuario). Estos permisos se solicitan como permisos delegados (no de aplicacion), lo que significa que cada usuario solo puede acceder a sus propios archivos en OneDrive, nunca a los de otros usuarios. Para compartir archivos entre miembros de un proyecto, se utilizan los enlaces de comparticion (sharing links) de OneDrive que permiten acceso temporal y revocable sin necesidad de compartir las credenciales del usuario.

# 7. Scopes Requeridos de Microsoft Graph API

Para que la integracion funcione correctamente, la aplicacion registrada en Azure AD debe solicitar los siguientes permisos delegados (scopes) al usuario durante el proceso de autenticacion OAuth 2.0. Estos permisos definen el alcance de las operaciones que ArchiFlow puede realizar en nombre del usuario sobre su cuenta de OneDrive. Es fundamental solicitar solo los permisos estrictamente necesarios para minimizar la superficie de ataque y generar mayor confianza en los usuarios al momento de aceptar los permisos de la aplicacion.

| Scope | Tipo | Descripcion | Uso en ArchiFlow |
|---|---|---|---|
| Files.ReadWrite.All | Delegado | Lectura y escritura de archivos del usuario | Subir, descargar, eliminar, renombrar archivos |
| Files.Read.All | Delegado | Lectura de metadatos de archivos | Listar archivos, buscar, obtener thumbnails |
| User.Read | Delegado | Informacion basica del perfil | Obtener nombre y email del usuario autenticado |
| Sites.ReadWrite.All | Delegado | Acceso a sitios de SharePoint | Almacenamiento en SharePoint si se habilita |

Tabla 7: Scopes OAuth requeridos para la integracion con Microsoft Graph API.

# 8. Estrategia de Implementacion por Fases

La implementacion se divide en cuatro fases secuenciales, cada una con entregables claros y criterios de aceptacion definidos. Esta estrategia permite validar cada componente de forma independiente antes de integrarlo con el resto del sistema, reduciendo el riesgo de errores en cascada y facilitando la identificacion temprana de problemas. Cada fase incluye pruebas unitarias, pruebas de integracion y pruebas de aceptacion de usuario antes de pasar a la siguiente.

| Fase | Descripcion | Entregables | Esfuerzo |
|---|---|---|---|
| Fase 1 Configuracion | Registro en Azure AD, configuracion de scopes OAuth, integracion con Firebase Auth, y creacion del primer endpoint de prueba que lista los archivos del usuario en OneDrive. | App registrada en Azure AD, scopes configurados, endpoint /api/onedrive/test funcional, documentacion de configuracion. | 1 semana |
| Fase 2 Explorador | Desarrollo del explorador de archivos con navegacion por carpetas, busqueda, ordenamiento, breadcrums, y acciones CRUD (crear, renombrar, eliminar, descargar). | Componente FileExplorer, rutas API CRUD, sincronizacion con Firestore, pruebas unitarias. | 2 semanas |
| Fase 3 Galeria + Subida | Implementacion de la galeria de fotos con thumbnails y vista previa, componente de subida con drag-and-drop, barra de progreso, y carga por partes para archivos grandes. | Componente PhotoGallery, componente FileUpload, rutas API de galeria, soporte para batch upload. | 2 semanas |
| Fase 4 Permisos + Produccion | Implementacion de la matriz de permisos por rol, optimizacion de rendimiento, manejo de errores robusto, logging, y despliegue a produccion en Vercel. | Middleware de permisos, manejo de errores, pruebas E2E, documentacion de usuario, deploy a Vercel. | 1 semana |

Tabla 8: Plan de implementacion por fases con entregables y estimaciones.

# 9. Consideraciones Tecnicas

## 9.1 Gestion de Tokens y Refresh

Los tokens de acceso de Microsoft OAuth tienen una duracion limitada (generalmente 60 minutos). Para garantizar una experiencia fluida, es necesario implementar un mecanismo de refresh de tokens que se ejecute de forma transparente cuando un token expire. Firebase Auth gestiona automaticamente el refresh de su propio token, pero el token de acceso de Microsoft (accessToken) que se incluye en la respuesta de autenticacion necesita un manejo adicional. La estrategia recomendada es almacenar el refreshToken de Microsoft de forma segura en Firestore (encriptado) y utilizarlo desde las API Routes para obtener un nuevo accessToken cuando el actual haya expirado. Alternativamente, se puede configurar Firebase Auth para que utilice el flujo de "refresh token rotation" de Microsoft, que genera un nuevo refresh token cada vez que se utiliza, mejorando la seguridad.

## 9.2 Limitaciones de la API

Microsoft Graph API tiene varias limitaciones que deben tenerse en cuenta durante la implementacion. El tamano maximo de archivo para una carga simple (PUT) es de 4MB; para archivos mayores, es obligatorio utilizar la API de sesiones de carga (upload sessions) que permite cargar archivos de hasta 250GB en fragmentos. La API tiene un limite de solicitudes (throttling) de aproximadamente 10,000 solicitudes por cada 10 minutos por aplicacion, lo que requiere implementar mecanismos de retry con backoff exponencial y, opcionalmente, cache de respuestas para las operaciones mas frecuentes como la lista de archivos de una carpeta. La busqueda en OneDrive tiene un limite de resultados y no soporta busqueda por contenido en todos los tipos de archivo.

## 9.3 Rendimiento y Cache

Para optimizar el rendimiento de la aplicacion, se recomienda implementar una capa de cache intermedia utilizando Firestore como store de metadatos. Cada vez que se lista una carpeta de OneDrive, los metadatos de los archivos se almacenan en la coleccion onedrive_files de Firestore. Las consultas subsiguientes se realizan primero contra Firestore (que es significativamente mas rapido que llamar a Graph API) y se actualizan en segundo plano si los datos tienen mas de un tiempo configurado de antiguedad (stale-while-revalidate pattern). Los thumbnails de las fotos deben almacenarse en cache del lado del cliente utilizando el Cache API del navegador, con una politica de expiracion de 24 horas, para evitar descargar el mismo thumbnail multiples veces.

# 10. Resumen de Dependencias y Prerequisitos

Antes de iniciar la implementacion, es necesario completar los siguientes prerequisitos tecnicos y de configuracion. Estos pasos son fundamentales para que la integracion funcione correctamente y deben completarse en su totalidad antes de comenzar con la Fase 1 de desarrollo.

| Requisito | Descripcion | Estatus Actual |
|---|---|---|
| Cuenta Azure AD | Registrar aplicacion en Azure Portal con redirect URIs configurados para desarrollo y produccion. | Pendiente |
| Client ID / Secret | Obtener credenciales de la aplicacion registrada en Azure AD para configurar Firebase Auth. | Pendiente |
| Firebase Auth Microsoft | Habilitar proveedor de Microsoft en Firebase Console y configurar Client ID y Client Secret. | Parcialmente implementado |
| Scopes OAuth | Configurar los scopes Files.ReadWrite.All, Files.Read.All, User.Read en Azure AD. | Pendiente |
| Variables de entorno | Configurar AZURE_CLIENT_ID, AZURE_CLIENT_SECRET y NEXT_PUBLIC_MS_SCOPE en .env.local. | Pendiente |
| Firestore - Colecciones | Crear indices para las colecciones onedrive_files y onedrive_folders en Firestore. | Pendiente |

Tabla 9: Lista de prerequisitos y dependencias para la implementacion.